

Do You Code for 10 Users or 10 Million?

Scalability and Time Complexity

The Hobbyist

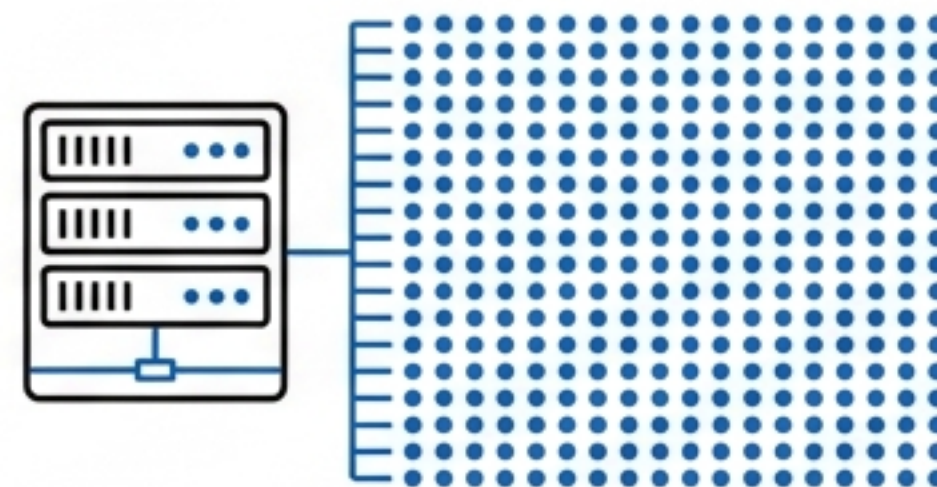


Student Project

$N = 10$ Users ✓

Works with any code. No crashes.

The Engineer



Global System

$N = 10,000,000$ Users ⚠

Inefficient code kills the system.

The Goal: Understand **Time Complexity** not as speed, but as survival at scale.

The Hardware Trap: Why 'Seconds' Are a Lie

High-End Server
(32 Cores, 128GB RAM)



Execution Time: 0.29 Seconds

Budget Laptop
(Dual Core, 4GB RAM)

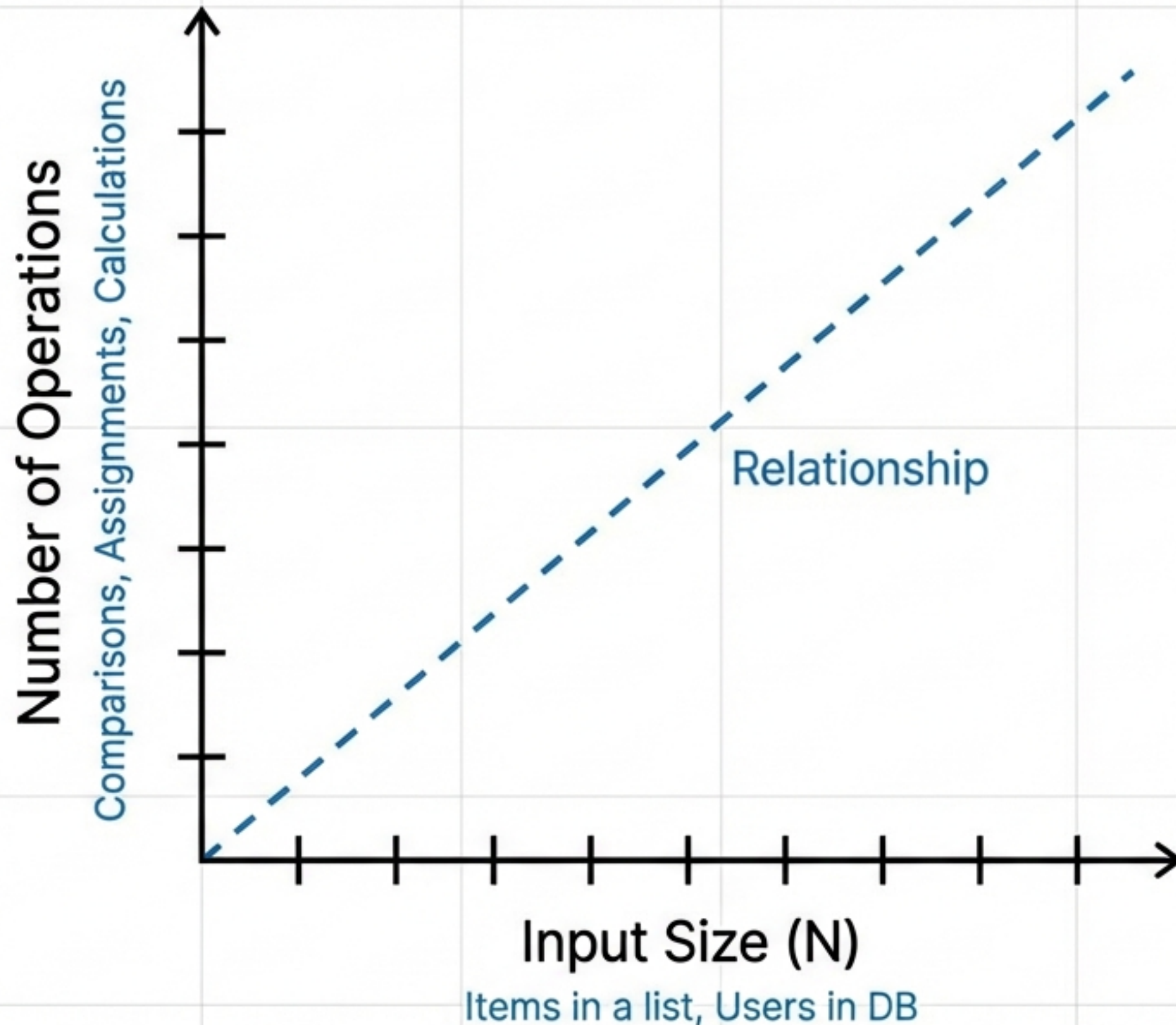


Execution Time: 12.00 Seconds

Same Code.
Different Hardware.

- **The Misconception:** Thinking **Time Complexity** = Wall Clock Time.
- The Reality: Run time depends on RAM, CPU, and background processes.
- **The Verdict:** We need a standard metric that ignores hardware differences entirely.

The Universal Metric: Operations vs. Input



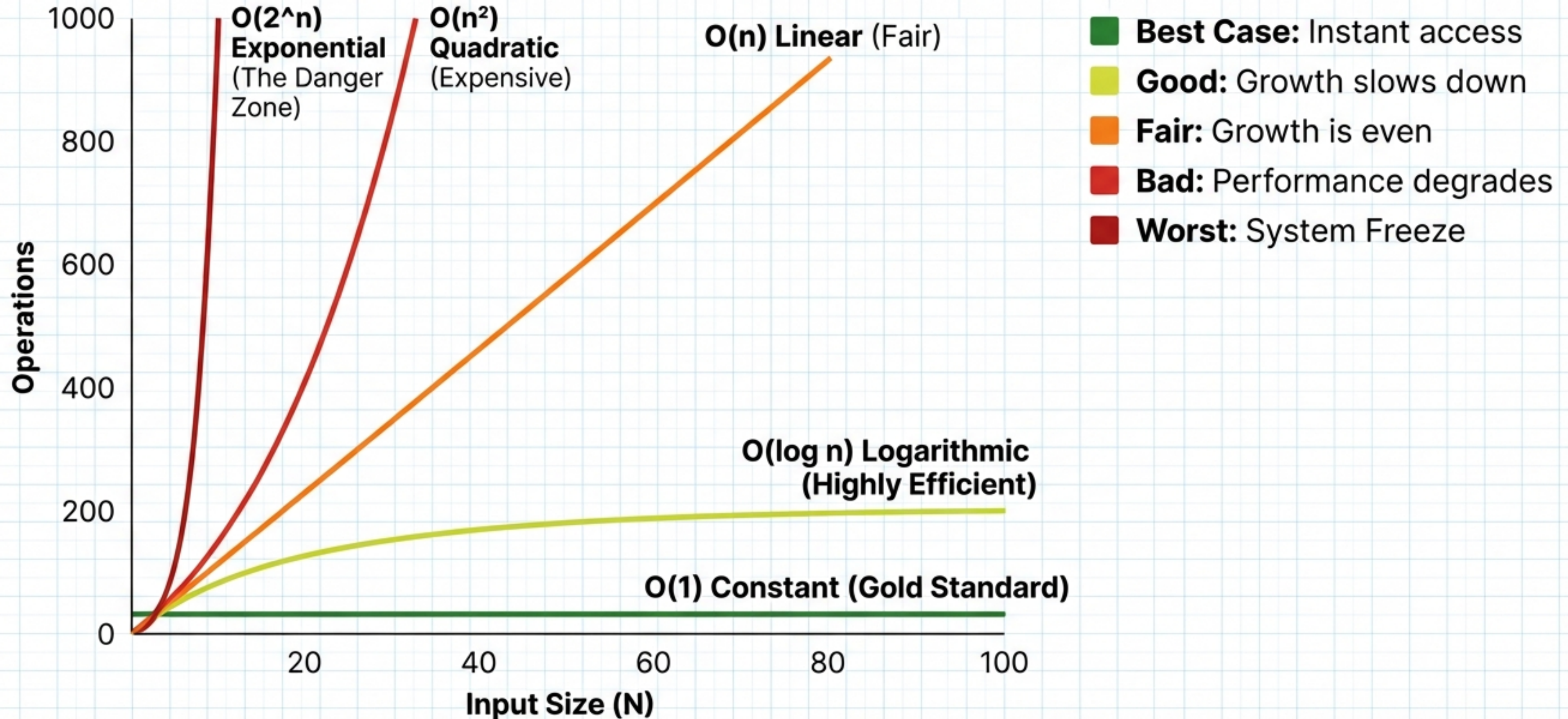
Time Complexity is the relationship between the **Input Size (N)** and the **Operations** the code must perform.

The Logic: If N increases, how much does the work (Y) increase?

The Metaphor: Operations are currency. We want to spend as few as possible to buy the result.

Formalized as: **Big O Notation**

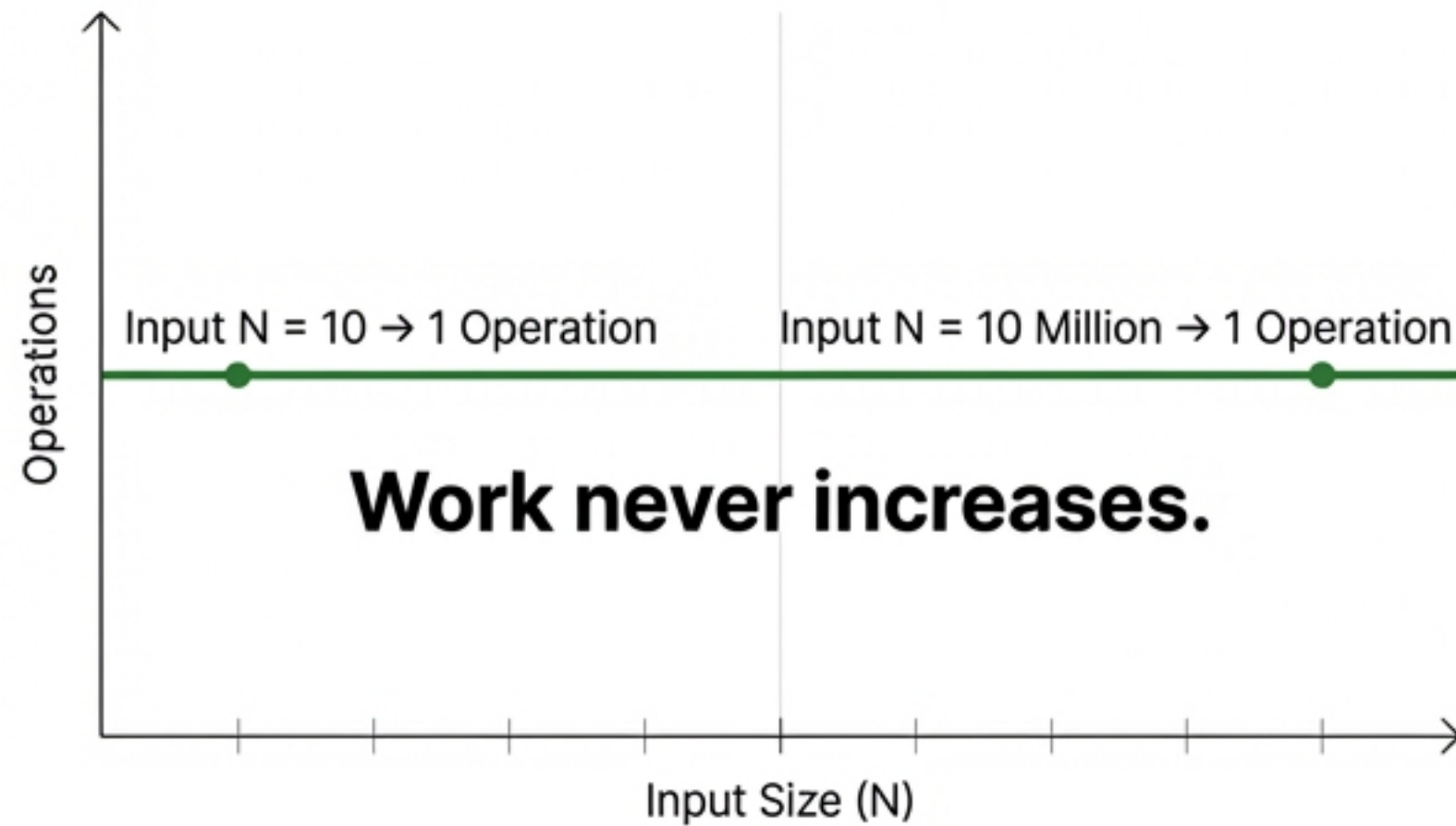
The Big O Hierarchy



Constant vs. Linear: The Baseline

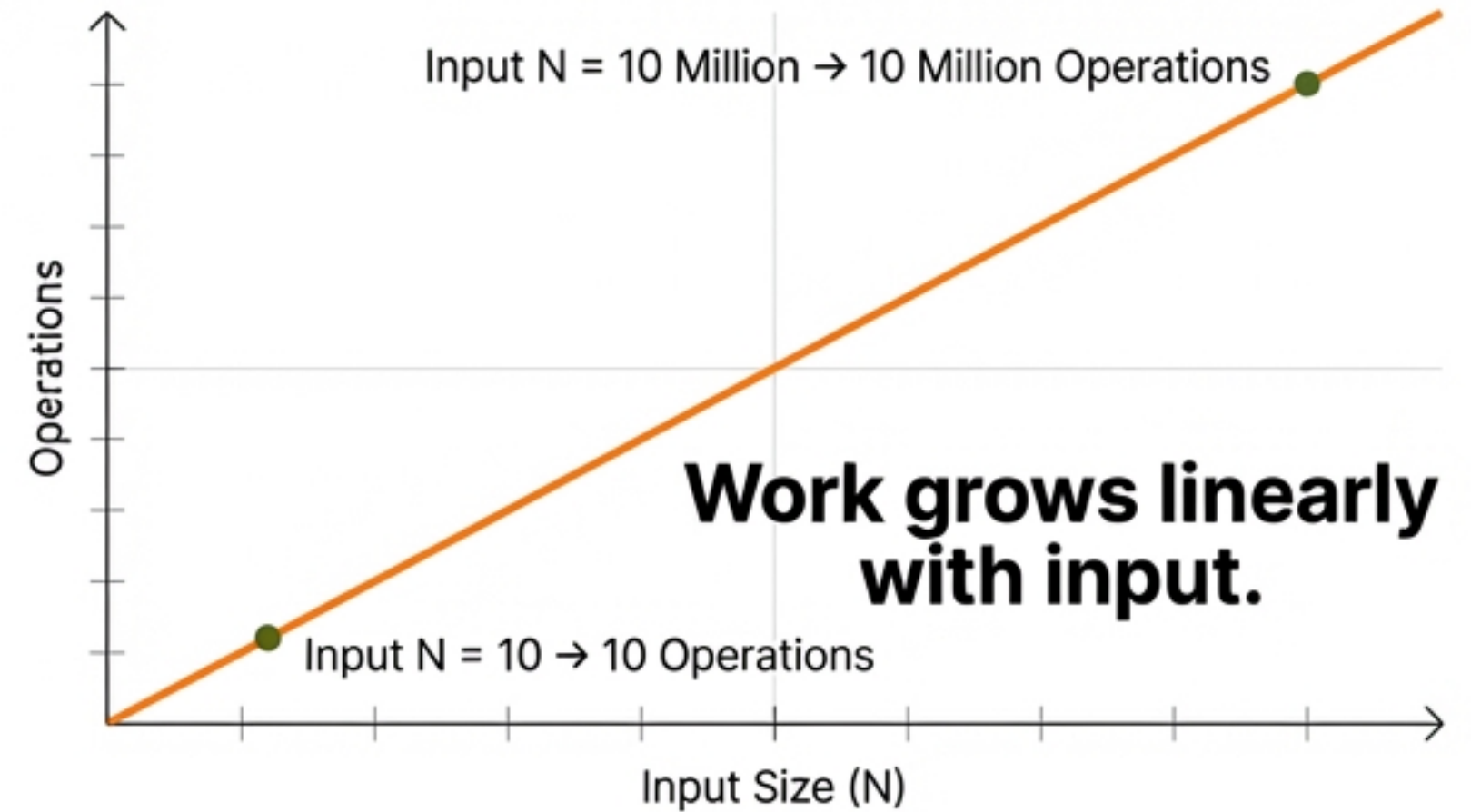
$O(1)$ Constant (Gold Standard)

```
def get_first(arr):  
    return arr[0]
```



$O(n)$ Linear (Fair)

```
def print_all(arr):  
    for item in arr:  
        print(item)
```



$O(n^2)$: The Nested Trap

```
# The Nested Loop
```

```
for i in range(n): # Outer Loop
```

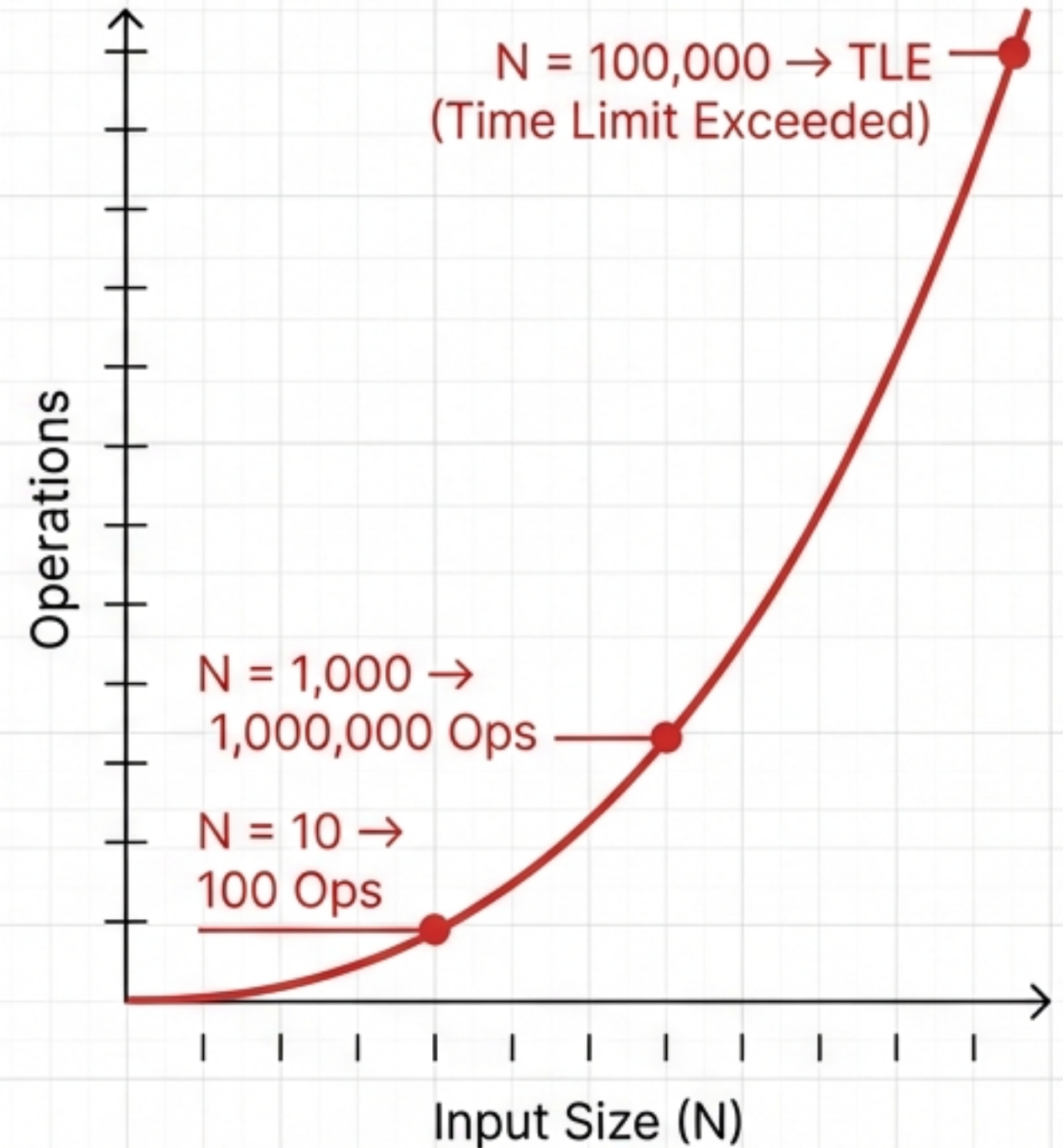
```
    for j in range(n): # Inner Loop
```

```
        print(i, j)
```

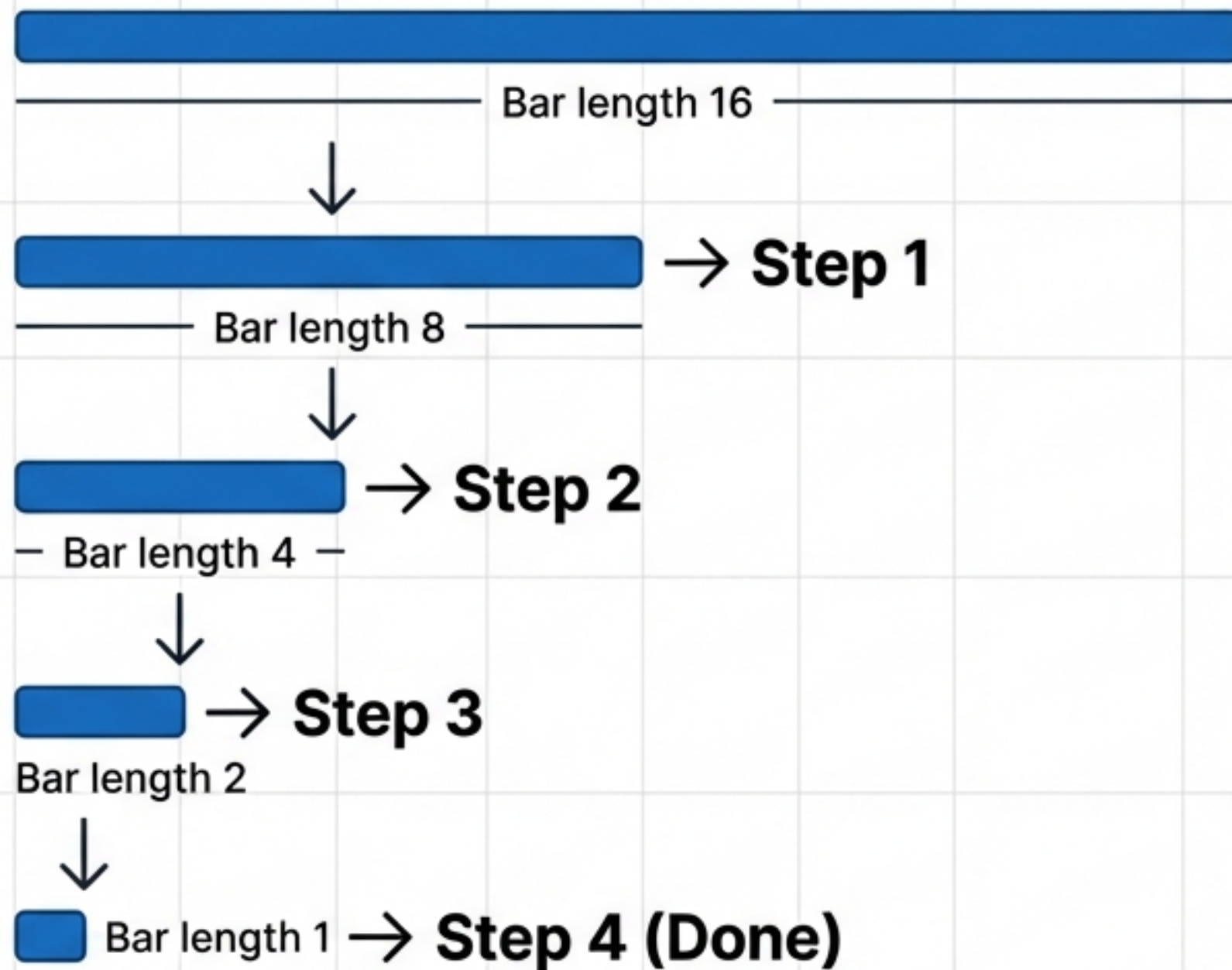
$N * N = N^2$

Operations

Common in **Bubble Sort** and **Matrix operations**.
Avoid for inputs $> 5,000$.



$O(\log n)$: The Power of Halving



Concept: Cut the problem in half at every step.

The Math: How many times can you divide N by 2 to get 1?

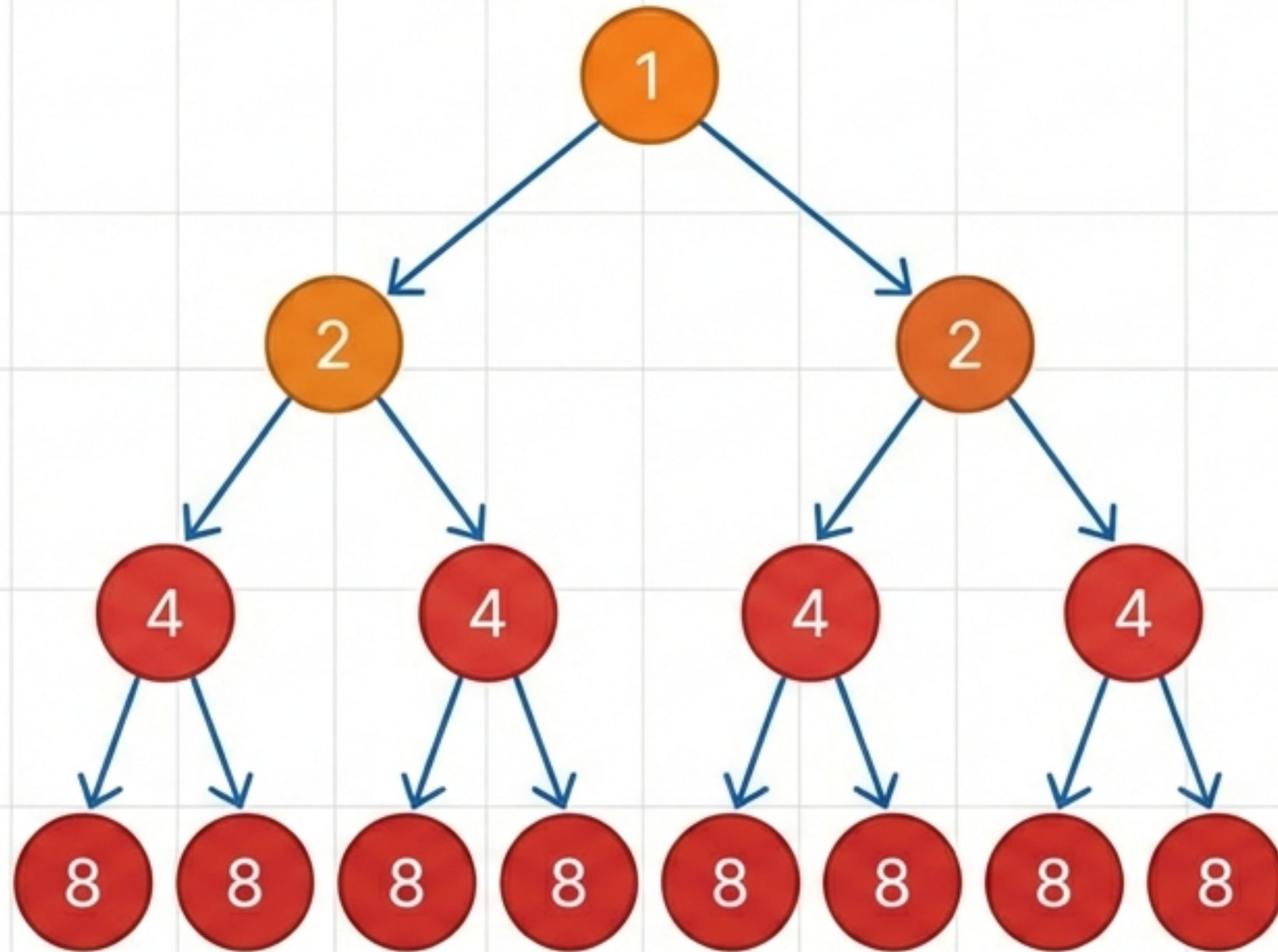
$N = 16 \rightarrow 4$ steps

$N = 1,000,000 \rightarrow$ **Only ~20 steps!**

Key Identifier Code

```
while n > 0:  
    n = n // 2
```


$O(2^n)$: The Danger Zone



The Pattern: Recursive algorithms that branch twice for every step (e.g., Fibonacci recursion).

The Growth: 1, 2, 4, 8, 16, 32, 64...

Data comparison:

$N = 10 \rightarrow \sim 1,000$ Operations

$N = 20 \rightarrow \sim 1,000,000$ Operations

The Rule: Only acceptable for tiny inputs ($N < 20$).

Reverse Engineering: Which Algorithm to Use?

Based on Input Size Constraints (Assuming 10^8 ops/sec)

Input Size (N)	Allowable Complexity	Algorithm Type
$N \leq 10$	$O(n!)$ or $O(2^n)$	Brute Force / Recursion
$N \leq 5,000$	$O(n^2)$	Nested Loops
$N \leq 1 \text{ Million } (10^6)$	$O(n)$ or $O(n \log n)$	Linear / Sorting
$N > 1 \text{ Billion } (10^{12})$	$O(\log n)$ or $O(1)$	Binary Search / Math

Strategy: Look at the problem's constraints FIRST, then select your algorithm.

Case Study: LeetCode 50 - Pow(x, n)

50. Pow(x, n)

Medium Difficulty

Implement $\text{pow}(x, n)$, which calculates x raised to the power n (x^n).

Example 1:

Input: $x = 2.00$, $n = 10$ | **Output:** 1024.00

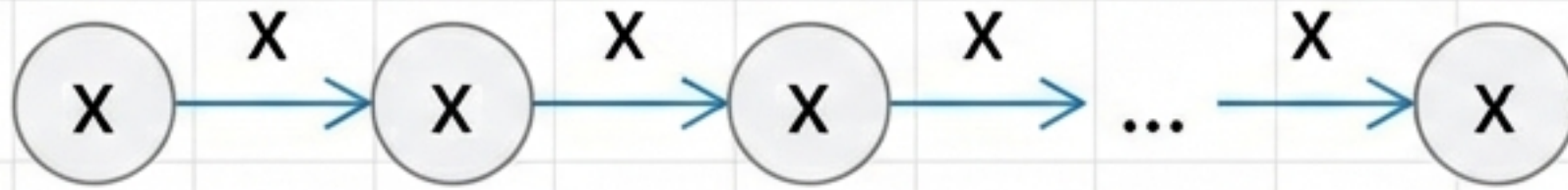
Example 2:

Input: $x = 2.10$, $n = 3$ | **Output:** 9.261

Constraints:

- $-100.0 < x < 100.0$
- $-2^{31} \leq n \leq 2^{31} - 1$
- Note: n is an integer, n can be negative.

The Naive Approach: Brute Force



N operations (Linear)

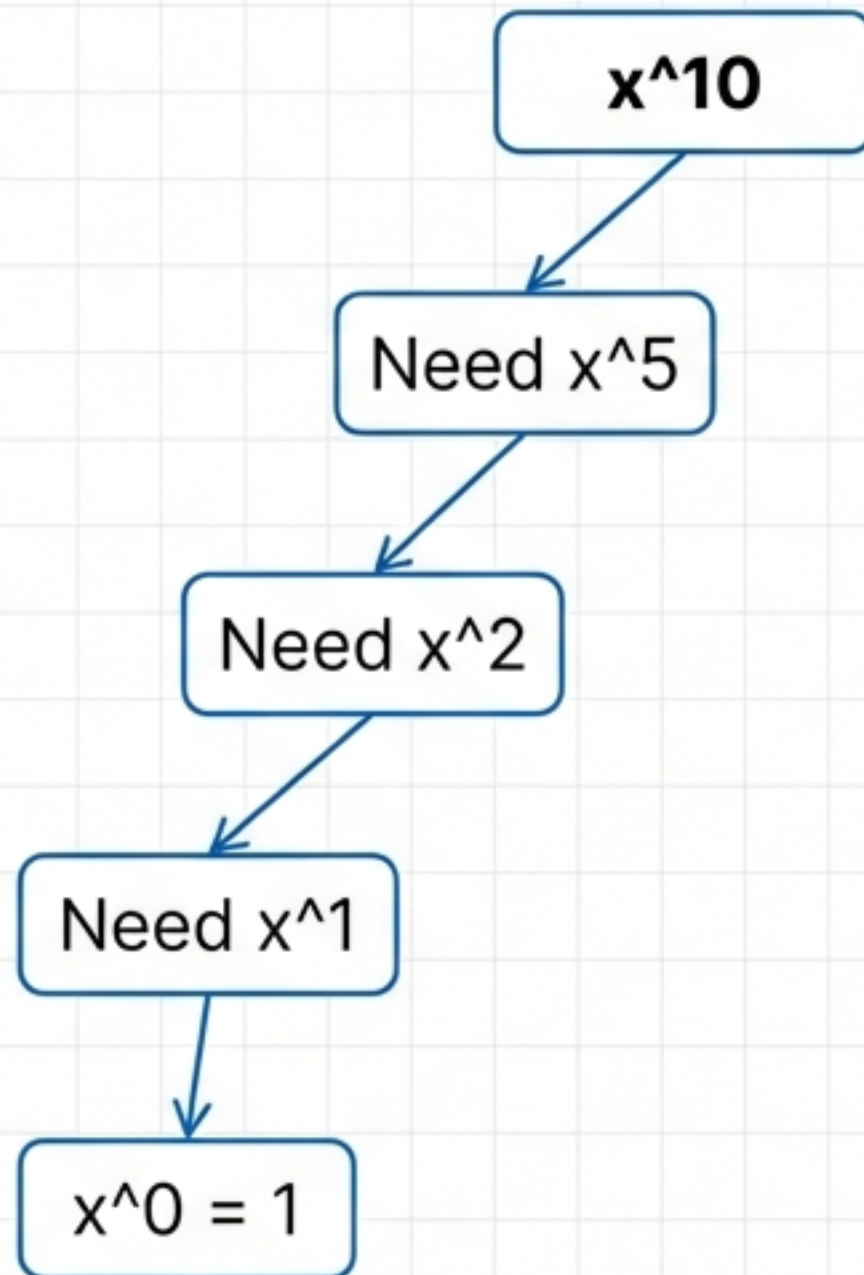
```
ans = 1
for i in range(n):
    ans = ans * x
```

Complexity: $O(n)$

The Failure: If $n = 2,147,483,647$ (Max Integer), the loop runs ~2 billion times.

TLE: Time Limit Exceeded

The Optimized Approach: Divide & Conquer



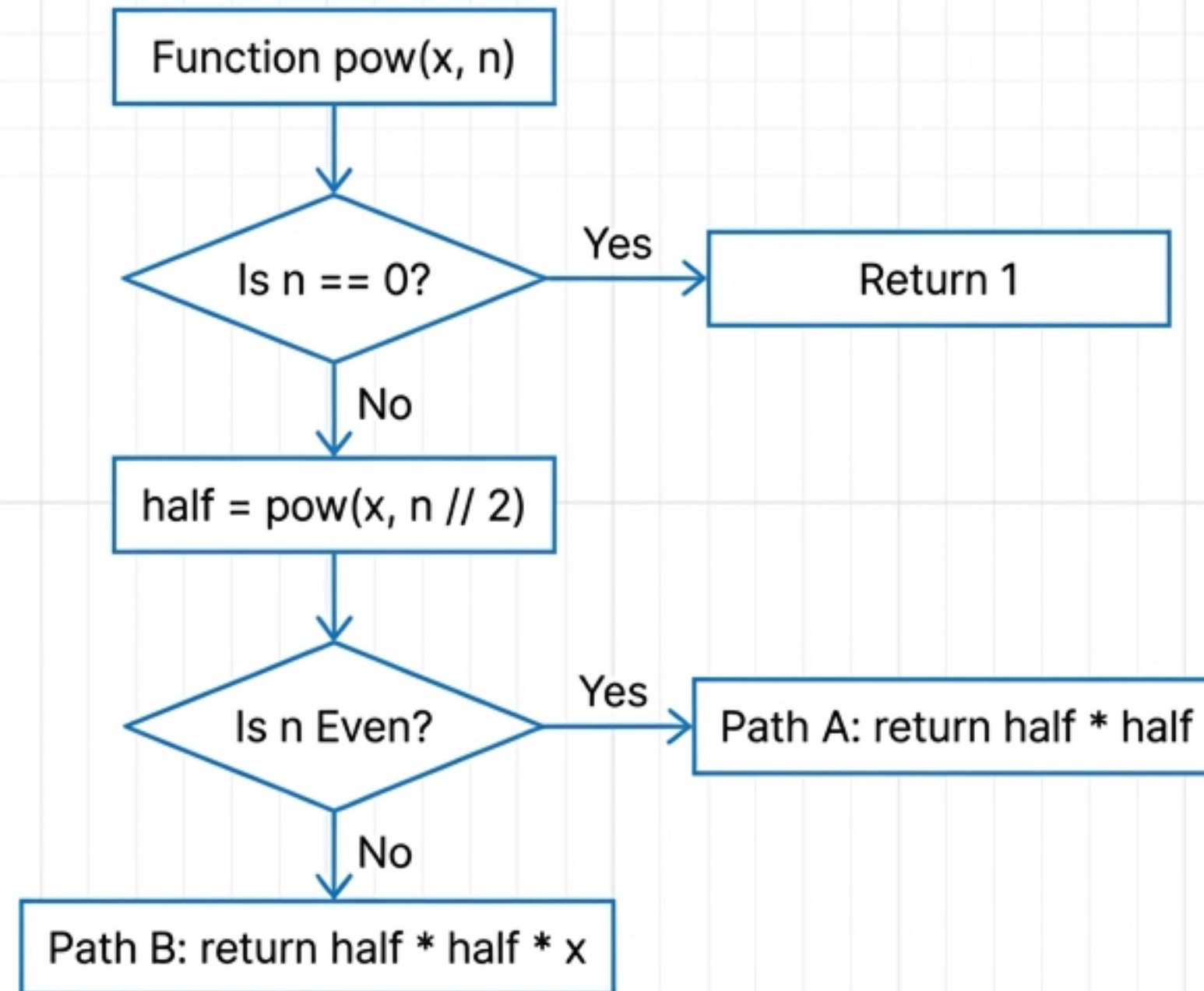
$$x^{10} = (x^5) * (x^5)$$

$$x^5 = x * (x^2) * (x^2)$$

Summary

- Technique: **Recursion**
- Complexity: **$O(\log n)$**
- Impact:
For **$n = 1024$** ,
Brute Force = **1024** steps.
Divide & Conquer = **10 steps.**

The Recursive Logic: Even vs. Odd



Even Example (n=10): $(2^5) * (2^5)$

Odd Example (n=5): $(2^2) * (2^2) * 2$

Handling Edge Cases: Negative Powers

$$x^{-n} = \frac{1}{x^n}$$

```
class Solution:
    def myPow(self, x: float, n: int) -> float:
        if n < 0:
            x = 1 / x
            n = -n # make n positive

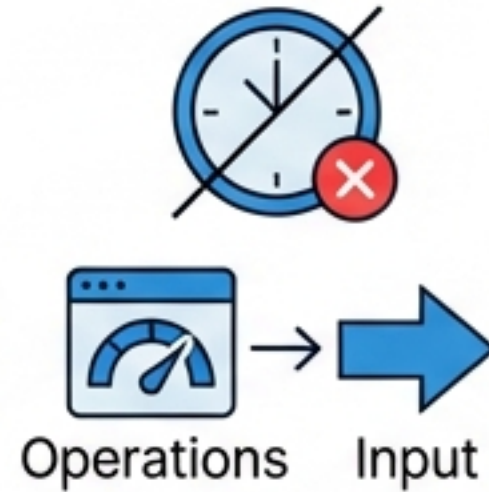
        return self.solve(x, n)
```

- Problem: n can be negative (e.g., $2^{-2} = 0.25$).
- Solution: **Invert x to (1/x) and make n positive.**
- Base Case reminder: If **n=0**, always return **1**.

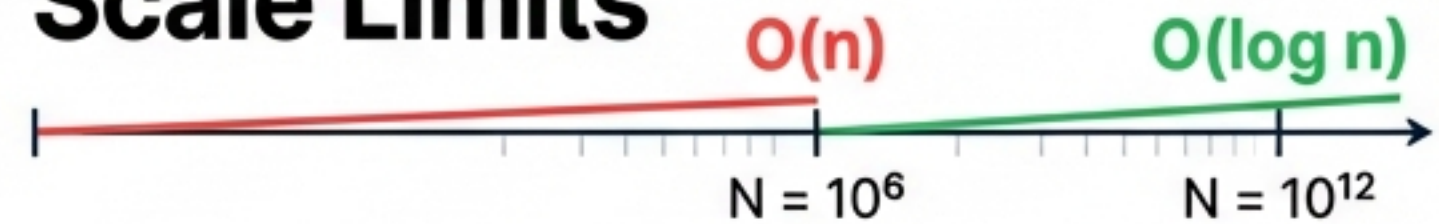
The Efficiency Cheat Sheet

The Golden Rule

- Don't Trust the Clock.
- Metric: **Operations** vs Input.



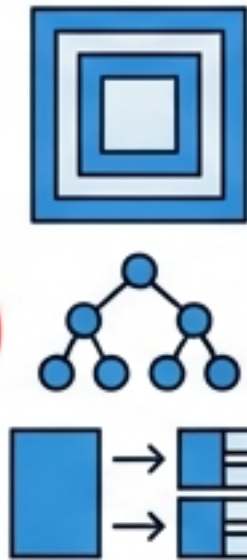
Scale Limits



- $N = 10^6 \rightarrow O(n)$ required.
- $N = 10^{12} \rightarrow O(\log n)$ required.

Code Smells

- Nested Loops $\rightarrow O(n^2)$
- Recursive Branching $\rightarrow O(2^n)$
- Halving Input $\rightarrow O(\log n)$



The Lesson

- We optimize code not just for speed, but for survival.

